

Facultad de
INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

I118 Taller de Proyecto II

# ParkFinder: Sistema de localización de vehículos en estacionamiento

*Informe de Avance de octubre - Proyecto G2*


Seijo, Gerónimo - 01859/7

seijo.geronimo@alu.ing.unlp.edu.ar

Maranzana, Valentina - 01917/9

valenmaranzanaa@gmail.com

Perez Del Llano, Thiago Tomas - 03078/3

thiago.tomas.pdll@gmail.com

Universidad Nacional de La Plata


9 de octubre de 2026

# Índice

1. Descripción General del Proyecto ..... 1
2. Materiales y Presupuesto ..... 2
3. Esquema Gráfico del Proyecto ..... 3
4. Grado de Avance, Problemas y Soluciones ..... 7
   4.1. *Hardware* probado ..... 10
   4.2. Planificación del Avance de noviembre ..... 11
   4.3. Documentación de Código ..... 12
      4.3.1. Código que controla el *hardware* ..... 12
      4.3.2. Código de comunicaciones ..... 13
      4.3.3. Sistema web ..... 15
      4.3.4. *Software* de la unidad central ..... 15
   4.4. Problemas y soluciones ..... 16
5. Documentación Relacionada ..... 17
Bibliografía ..... 19

# Índice de figuras

Figura 1 Esquemático del sistema de localización. . . . . 4
Figura 2 Conexionado del ESP32 con el sensor HC-SR04 y el LED bicolor de la plaza 1. . . . . 5
Figura 3 Montaje del ESP32 con sensores y LED, vista superior y lateral. . . . . 5
Figura 4 Captura de la cámara C270 con las zonas de las plazas y las patentes leídas. . . . . 10
Figura 5 Maqueta, cámara con soporte provisorio y Raspberry Pi con pendrive. . . . . 18

# Índice de tablas

Tabla 1 Componentes del prototipo, estado y costo aproximado. . . . . 2
Tabla 2 Conexiones de la placa de desarrollo con los dispositivos. . . . . 6
Tabla 3 Grado de avance por parte del proyecto. . . . . 7
Tabla 4 Cumplimiento de lo planificado para el Informe de Avance de octubre. . . . . 7
Tabla 5 Comparación de motores de OCR en la PC y en la Raspberry Pi. . . . . 9
Tabla 6 Encuadre de la cámara C270 según la distancia a las patentes. . . . . 9
Tabla 7 Estado de prueba del hardware. . . . . 10
Tabla 8 Tópicos MQTT acordados entre la Raspberry Pi y el ESP32. . . . . 14
Tabla 9 Prueba de integración: entradas registradas a partir de los avisos del ESP32. . . . . 15
Tabla 10 Problemas encontrados y soluciones o alternativas. . . . . 16

## 1. Descripción General del Proyecto

Localizar el propio vehículo dentro de un estacionamiento de gran capacidad es un problema frecuente y de impacto directo en la experiencia del usuario. Los sistemas de guía convencionales informan únicamente la disponibilidad global de lugares, sin resolver la ubicación precisa de un vehículo ya estacionado.

ParkFinder es un sistema que asocia automáticamente la patente de cada vehículo con la plaza que ocupa y permite su posterior consulta. Cuando un vehículo ocupa una plaza, un sensor ultrasónico HC-SR04 [1] detecta el cambio y un microcontrolador ESP32 [2] lo informa mediante el protocolo *Message Queuing Telemetry Transport* (MQTT) [3]. La unidad central, una Raspberry Pi 3 [4], captura entonces una imagen, reconoce la patente mediante reconocimiento óptico de caracteres (OCR) y registra la asociación en una base de datos SQLite [5]. Un LED bicolor señaliza el estado de cada plaza, y una interfaz web, pensada como tótem de consulta, permite al conductor recuperar la ubicación de su vehículo ingresando la patente. El desarrollo se materializa en una maqueta de dos plazas, escalable.

**Objetivo general:** diseñar e implementar un sistema de localización de vehículos en un estacionamiento que asocie de forma automática la patente de cada vehículo con la plaza que ocupa, y permita su consulta mediante una interfaz web.

**Objetivos primarios:**
- Reconocer la patente de un vehículo entrante mediante procesamiento de imagen.
- Detectar el estado de ocupación de cada plaza mediante sensado ultrasónico.
- Registrar de forma persistente la asociación entre patente y plaza.
- Señalizar el estado de cada plaza mediante indicación luminosa.
- Proveer una interfaz web de consulta de ubicación por patente.

**Objetivos secundarios:**
- Mantener una latencia de respuesta acotada, adecuada a una demostración fluida.
- Adoptar una arquitectura modular que facilite el mantenimiento y la evolución del sistema.
- Contemplar la escalabilidad del prototipo a un número mayor de plazas.

**Requerimientos funcionales:**
- **RF01:** capturar la imagen del vehículo entrante, disparada por el evento de ocupación de una plaza.
- **RF02:** detectar la presencia de un vehículo en cada plaza mediante sensor ultrasónico HC-SR04.
- **RF03:** reconocer los caracteres de la patente y validarlos según el formato vehicular vigente.

- **RF04:** asociar la patente reconocida con la plaza correspondiente y registrarla de forma persistente.
- **RF05:** reflejar el estado de cada plaza (libre u ocupada) mediante un LED bicolor.
- **RF06:** responder las consultas de ubicación por patente a través de la interfaz web.

**Requerimientos no funcionales:**

- **RNF01 (latencia de respuesta):** reflejar el cambio de estado físico en el registro y en la señalización con una demora acotada, cuyo valor se establece y valida sobre la maqueta.
- **RNF02 (robustez):** tolerar lecturas de OCR fallidas y ruido en el sensado sin degradar la consistencia del registro.
- **RNF03 (consistencia):** mantener coherentes el estado señalizado, el sensado y el registro en la base de datos.
- **RNF04 (mantenibilidad):** permitir sustituir componentes, por ejemplo el motor de OCR, sin afectar al resto del sistema.
- **RNF05 (eficiencia):** ajustar el procesamiento a las capacidades de la plataforma embebida.

**Lo propuesto y lo que cambió.** El Plan de Proyecto presentado el 29 de agosto de 2026 proponía este mismo alcance, con Tesseract [6] como motor de OCR inicial, sujeto a validación frente a EasyOCR [7] y PaddleOCR [8], y dejaba abierta la elección entre una cámara USB y el módulo de cámara de la Raspberry Pi. En la ejecución, el alcance y los requerimientos se mantuvieron y se tomaron tres decisiones: se adoptó una cámara web USB (Logitech C270), el sistema operativo de la Raspberry Pi se aloja en un pendrive USB en lugar de una tarjeta microSD, y el LED de cada plaza lo maneja el propio ESP32 según su sensor HC-SR04, para responder sin depender de la red. La evaluación de los motores de OCR confirmó a Tesseract (Sección 4). El detalle de cada punto se explica en las secciones siguientes.

## 2. Materiales y Presupuesto

La Tabla 1 lista los componentes del prototipo con su cantidad, su origen (cátedra o grupo), su costo aproximado y el enlace donde se consultó el precio.

| Componente | Cant. | Origen | Costo aprox. (pesos) | Referencia |
|---|---|---|---|---|
| Raspberry Pi 3 B+ | 1 | Provista por la cátedra | $ 122.550 | https://www.mercadolibre.com.ar/microcontrolador-arm-raspberry-pi-3--b-element14-con-bluetooth-y-wifi-5ghz/p/MLA32953926 |

| Componente | Cant. | Origen | Costo aprox. (pesos) | Referencia |
|---|---|---|---|---|
| Pendrive USB 32 GB | 1 | Provisto por la cátedra | $ 20.690 | https://www.mercadolibre.com.ar/pendrive-sandisk-cruzer-blade-32gb-20-negro-y-rojo/p/MLA6078536 |
| Cámara web USB Logitech C270 | 1 | Adquirida por la cátedra | $ 46.199 | https://www.mercadolibre.com.ar/p/MLA18931576 |
| ESP32 | 1 | Aportado por el grupo | $ 16.276 | https://www.mercadolibre.com.ar/placa-de-desarrollo-dual-core-wifi-ble-42-240-mhz/p/MLA2038887439 |
| Sensor ultrasónico HC-SR04 | 2 | Provisto por la cátedra | $ 6.406 (por 2 unidades) | https://www.mercadolibre.com.ar/2-x-sensor-ultrasonico-hc-sr04-para-arduino-pic-robotica/p/MLA2043575349 |
| LED bicolor | 2 | Adquirido por el grupo | $ 126 c/u | https://candy-ho.com/producto/led-rojo-y-verde-5mm-bicolor-difuso/ |
| Resistencia de 220 Ω | 2 | Adquirida por el grupo | $ 314 c/u | https://candy-ho.com/producto/10x-resistencia-1-4-5-carbon-220-ohm/ |
| Resistencia de 1 kΩ | 2 | Adquirida por el grupo | $ 410 c/u | https://candy-ho.com/producto/10x-resistencia-1-4-5-carbon-1k-ohm/ |
| Resistencia de 2 kΩ | 2 | Adquirida por el grupo | $ 410 c/u | https://candy-ho.com/producto/10x-resistencia-1-4-5-carbon-2k-ohm/ |
| Fuente para protoboard MB102 | 1 | Aportada por el grupo | $ 3.724 | https://candy-ho.com/producto/fuente-para-protoboard-5v-3-3v-salida-usb-mb102-arduino-xd42/ |
| Materiales de la maqueta (soporte, plazas y montaje de la cámara) | - | Sin definir | - | - |

Tabla 1: Componentes del prototipo, estado y costo aproximado.

No se utilizan servicios en la nube: el *broker* MQTT, la base de datos y el servidor web residen en la Raspberry Pi.

## 3. Esquema Gráfico del Proyecto

La Figura 1 presenta el esquemático del sistema. Todo el procesamiento reside en la Raspberry Pi: el *broker* Mosquitto [9] recibe los avisos de ocupación que publica el ESP32; el *backend* en Python los atiende y dispara la captura de la cámara; el OCR (OpenCV [10] y Tesseract) reconoce la patente sobre la imagen; el resultado se registra en SQLite, y el servidor web

consulta esa base para atender al tótem. En el extremo de sensado, el ESP32 lee el HC-SR04 de cada plaza y enciende el LED bicolor correspondiente.

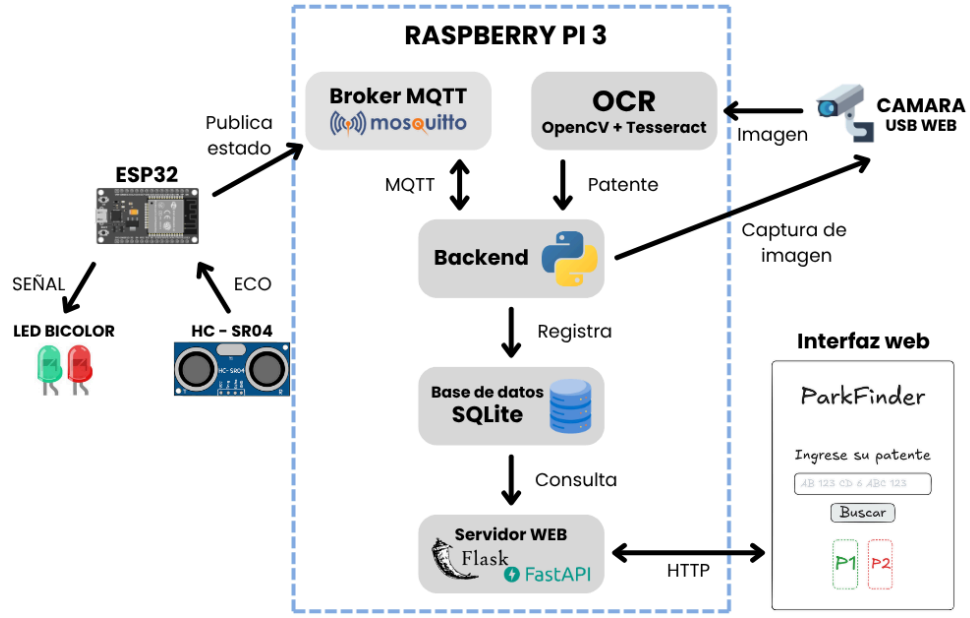



Figura 1: Esquemático del sistema de localización.

**Funcionamiento (caso de uso: ingreso de un vehículo):**

1. El HC-SR04 de la plaza detecta la ocupación y el ESP32 publica `ocupada` en el tópico `parkfinder/plaza/N`, donde N es el número de plaza.
2. La Raspberry Pi, suscripta a esos tópicos, captura la imagen con la cámara fija.
3. Recorta la zona calibrada de la plaza N, localiza la patente dentro de ella y la reconoce con Tesseract.
4. Corrige los caracteres confundibles según su posición y valida el formato de la patente.
5. Registra la asociación (patente, plaza y hora de ingreso) en SQLite.
6. Al liberarse la plaza, el ESP32 publica `libre` y se registra la hora de egreso.
7. El tótem consulta la plaza de una patente sobre las estadías abiertas.

**Esquema de conexiones físicas:** La Figura 2 muestra el conexionado del ESP32 para la plaza 1: el HC-SR04 se alimenta desde el pin VIN, su señal Trigger va al pin de entrada/salida de propósito general (GPIO) 13 y su señal Echo llega al GPIO 12 a través de un divisor de tensión, con una resistencia de 1 kΩ entre el Echo y el GPIO 12 y otra de 2 kΩ entre el GPIO 12 y GND, que reduce los 5 V del sensor al nivel de 3,3 V que admite el ESP32. El LED bicolor de dos pines se conecta entre el GPIO 25 y el GPIO 26, con una resistencia limitadora de 220 Ω en serie: según cuál de los dos pines esté en alto, el LED enciende en rojo o en verde. La plaza 2 replica el mismo circuito con el Trigger en el GPIO 27, el Echo en el GPIO 34 (un pin solo de entrada)

y el LED en los GPIO 32 y 33, según detalla la Tabla 2, que incluye además la alimentación y las conexiones de la Raspberry Pi.

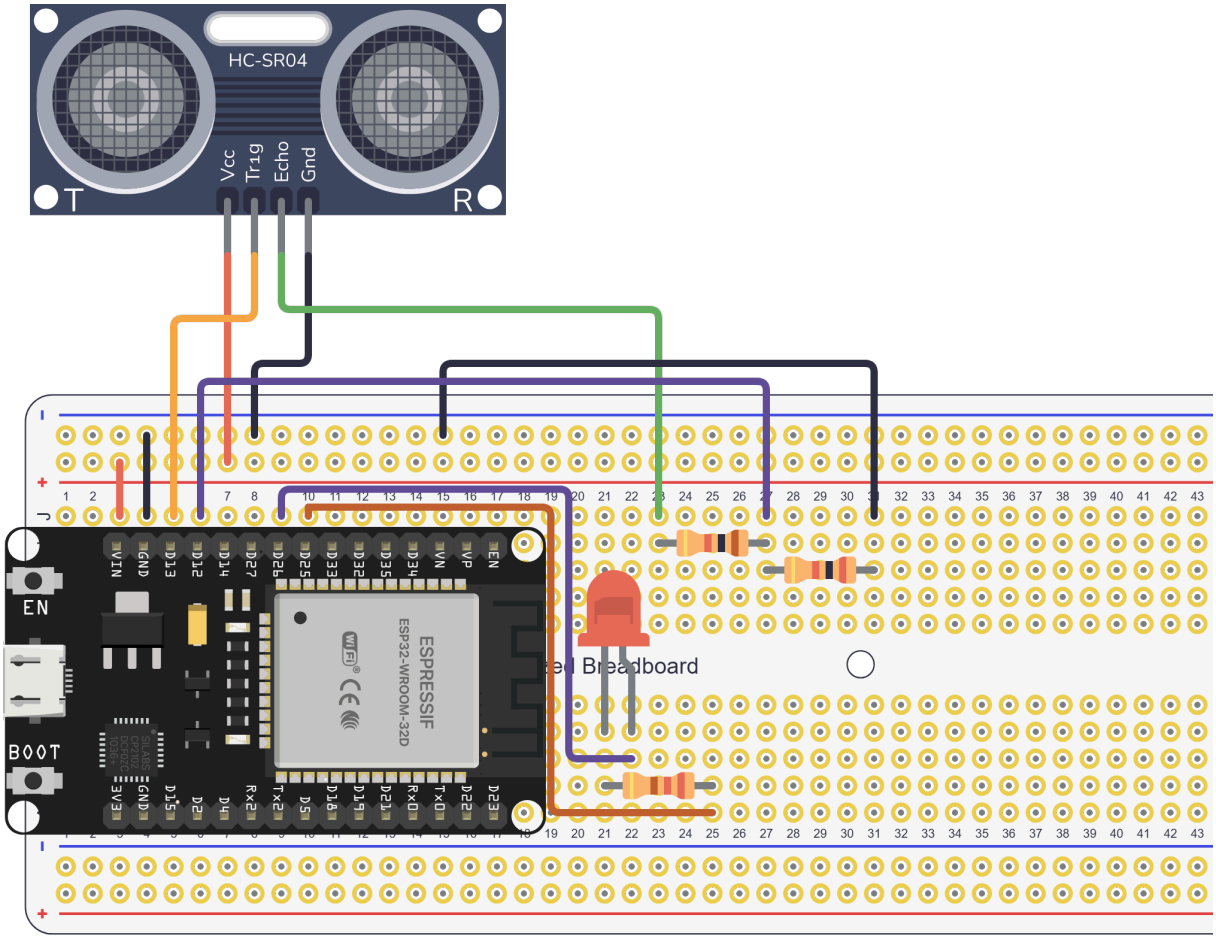



Figura 2: Conexionado del ESP32 con el sensor HC-SR04 y el LED bicolor de la plaza 1.

La Figura 2 es ilustrativa y muestra el circuito de una sola plaza para no recargar el esquema; el montaje real, con los dos sensores y los dos LED sobre el protoboard, se observa en la Figura 3.

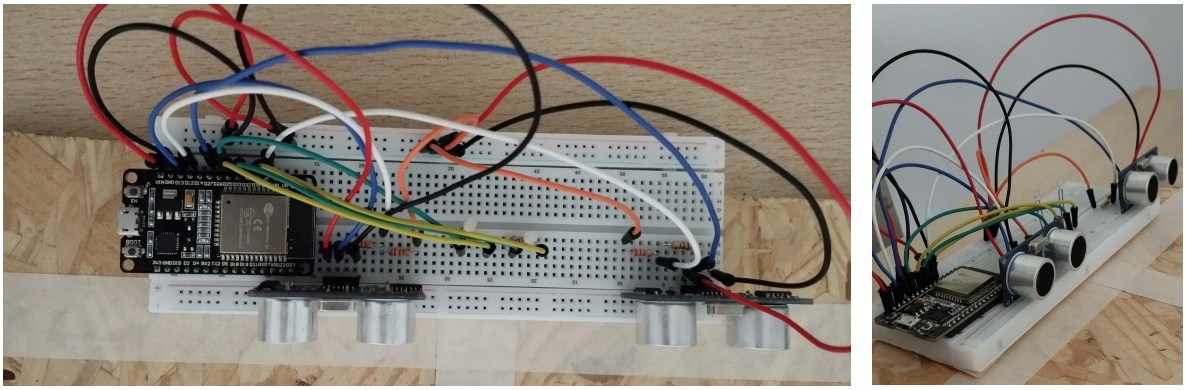


Figura 3: Montaje del ESP32 con sensores y LED, vista superior y lateral.

| Terminal | Se conecta a | Observación |
|---|---|---|
| **Sensor ultrasónico HC-SR04, plaza 1** (Figura 2) | | |
| VCC | VIN del ESP32 | Alimentación de 5 V. |
| GND | GND del ESP32 | Masa común. |
| TRIG | GPIO 13 | Salida del ESP32 que dispara la medición. |
| ECHO | GPIO 12, a través de un divisor de tensión | Resistencia de 1 kΩ entre ECHO y el GPIO 12, y de 2 kΩ entre el GPIO 12 y GND: reduce los 5 V del sensor a 3,3 V. |
| **Sensor ultrasónico HC-SR04, plaza 2** | | |
| VCC | VIN del ESP32 | Alimentación de 5 V. |
| GND | GND del ESP32 | Masa común. |
| TRIG | GPIO 27 | Salida del ESP32 que dispara la medición. |
| ECHO | GPIO 34, a través de un divisor de tensión | Mismo divisor que en la plaza 1: 1 kΩ y 2 kΩ. El GPIO 34 es solo de entrada y no cumple ninguna función durante el arranque. |
| **LED bicolor, plaza 1** (Figura 2) | | |
| Terminal 1 (rojo) | GPIO 25, con una resistencia de 220 Ω en serie | Con el GPIO 25 en alto y el 26 en bajo, el LED enciende en rojo. |
| Terminal 2 (verde) | GPIO 26 | Con el GPIO 26 en alto y el 25 en bajo, el LED enciende en verde. |
| **LED bicolor de 2 pines, plaza 2** | | |
| Terminal 1 (rojo) | GPIO 32, con una resistencia de 220 Ω en serie | Mismo esquema que en la plaza 1. |
| Terminal 2 (verde) | GPIO 33 | Mismo esquema que en la plaza 1. |
| **Alimentación** | | |
| ESP32 | Puerto micro-USB | 5 V por USB, desde un cargador de celular o desde la PC al programarlo. El pin VIN entrega esa tensión a los sensores (4,7 a 4,9 V medidos) y los LED se alimentan desde los GPIO a 3,3 V. |
| Raspberry Pi 3 | Puerto micro-USB | Cargador de celular de 5 V / 3 A. |
| **Cámara, almacenamiento y comunicación** | | |
| Cámara C270 | Puerto USB de la Raspberry Pi | Transmite el video de las plazas. |
| Pendrive USB | Puerto USB de la Raspberry Pi | Aloja el sistema operativo. |
| ESP32 con Raspberry Pi | Red WiFi local (punto de acceso de un teléfono móvil) | Mensajes MQTT hacia el *broker* de la Raspberry Pi, que el ESP32 busca por su nombre en la red (`parkfinder.local`). |

Tabla 2: Conexiones de la placa de desarrollo con los dispositivos.

Las conexiones de la Raspberry Pi se limitan a la alimentación y a sus puertos USB, donde se enchufan la cámara y el pendrive del sistema. El ESP32 y la Raspberry Pi no tienen ningún cableado entre sí: se comunican únicamente a través de la red WiFi del teléfono móvil.

## 4. Grado de Avance, Problemas y Soluciones

La Tabla 3 resume el estado de avance de cada parte del proyecto.

| **Parte del proyecto** | **Estado** |
|---|---|
| Reconocimiento de patentes (OCR) | Funcional: motor elegido y medido en la Raspberry Pi, y recorte por plaza, validación y captura funcionando con la cámara C270. Resta anclar la cámara y repetir la calibración de las zonas. |
| Persistencia (SQLite) | Funcional: modelo de datos y asociación patente-plaza implementados y probados. |
| Mensajería MQTT | Funcional: *broker* Mosquitto, suscriptor y publicación desde el ESP32, probados de punta a punta con la cámara C270 a través de la red local de un teléfono móvil. |
| Sensado, LED y ESP32 | Funcional: sensado con histéresis y confirmación, señalización de los LED y publicación MQTT, probados en las dos plazas. |
| Servicio web y tótem | Planificado para noviembre. |

Tabla 3: Grado de avance por parte del proyecto.

Para cada parte, lo básico está desarrollado y funcional; lo que resta son mejoras de robustez y ajustes que se abordan en el Avance de noviembre, junto con el servicio web y el tótem de consulta.

En síntesis, las funciones básicas del sistema están desarrolladas y funcionan de punta a punta con el ESP32 real: al ocupar una plaza, el sensor avisa por MQTT, la Raspberry Pi captura la imagen, lee la patente y la registra. Para noviembre queda mejorar la robustez del reconocimiento y de la mensajería, y sumar el servicio web y el tótem.

El Plan de Proyecto fijó para octubre tres frentes: el reconocimiento de patentes, el sensado con la arquitectura de mensajería, y la persistencia. La Tabla 4 indica el cumplimiento de cada ítem.

| **Ítem del plan** | **Cumplido** | **Comentario** |
|---|---|---|
| Captura de imagen disparada por el evento de ocupación (RF01) | Sí | Con la cámara C270 en la Raspberry Pi, el aviso MQTT del ESP32 dispara la captura (5 de 5 entradas de la prueba de integración). |

| Ítem del plan | Cumplido | Comentario |
|---|---|---|
| Recorte de la región fija de cada plaza | Sí | Zonas calibradas con la cámara a 30 cm (Tabla 6). Falta repetir la calibración al anclar la cámara en la maqueta. |
| Reconocimiento de patente con Tesseract (RF03) | Sí | 97 de 144 imágenes del banco de pruebas en la Raspberry Pi; 4 de 4 patentes con la cámara C270, en formato anterior y Mercosur. |
| Evaluación comparativa de motores y ajuste del preprocesamiento (RNF04, RNF05) | Sí | Tres motores medidos en PC y en la Raspberry Pi; preprocesamiento con escala de grises y filtro de mediana (Tabla 5). |
| Validación del formato de patente por expresión regular (RF03) | Sí | Formato anterior y Mercosur, con corrección de caracteres confundibles según su posición. |
| Lectura del HC-SR04 y umbral de ocupación (RF02) | Sí | Probado en las dos plazas: histéresis (ocupa a 10 cm o menos, libera a más de 13 cm) y confirmación del cambio durante 1,5 s. |
| Control del LED bicolor (RF05) | Sí | Probado en las dos plazas: el ESP32 maneja cada LED localmente según su sensor (rojo ocupada, verde libre). |
| Integración del ESP32 con los sensores y los LED | Sí | Dos sensores y dos LED integrados y probados en el ESP32. |
| *Broker* Mosquitto y estructura de tópicos | Sí | Mosquitto en la Raspberry Pi, accesible desde la red local; tópicos `parkfinder/plaza/N`. |
| Publicación desde el ESP32 y suscripción desde la Raspberry Pi | Sí | El ESP32 publica en `parkfinder/plaza/N` por la red del teléfono móvil y la Raspberry Pi lo recibe: 5 de 5 entradas leídas correctamente. Desde que el vehículo llega hasta su registro transcurren ≈ 3,8 s (1,5 s de confirmación, 0,3 s de publicación y ≈ 2,0 s de procesamiento). |
| Verificación de la capacidad de la Raspberry Pi 3 con la mensajería (RNF05) | Sí | Con *broker* y suscriptor activos: Mosquitto 7,6 MB, suscriptor 61 MB y 728 MB de RAM disponibles; el OCR usa un solo núcleo. |
| Modelo y creación de la base de datos SQLite | Sí | Tablas `estadias` y `lecturas`. |
| Lógica de asociación patente-plaza (RF04) | Sí | Registro de ingreso y de egreso de cada plaza. |

Tabla 4: Cumplimiento de lo planificado para el Informe de Avance de octubre.

Con estos resultados quedan cumplidos los tres frentes que el Plan fijó para octubre. Lo que resta es de ajuste y no de funcionalidad: anclar la cámara de forma definitiva sobre la maqueta, repetir la calibración de las zonas.

La evaluación de motores se realizó sobre un banco de 144 imágenes (36 recortes de patentes en la tipografía FE-Schrift y 108 versiones degradadas), con el mismo código ejecutado en una PC y en la Raspberry Pi (Tabla 5).

| Motor | Aciertos PC | Tiempo por imagen PC | Aciertos Pi | Tiempo por imagen Pi | Pico de RAM Pi |
|---|---|---|---|---|---|
| **Tesseract** | 101 (70,1 %) | 0,457 s | **97 (67,4 %)** | **0,97 s** | **110 MB** |
| EasyOCR | 98 (68,1 %) | 1,099 s | 97 (67,4 %) | 17,2 s | 774 MB |
| PaddleOCR | **107 (74,3 %)** | 3,102 s | no ejecuta | - | - |

Tabla 5: Comparación de motores de OCR en la PC y en la Raspberry Pi.

En la PC, PaddleOCR es el más preciso (74,3 %), pero tarda 3,102 s por imagen: 6,8 veces más que Tesseract y 2,8 veces más que EasyOCR. En la Raspberry Pi 3 no ejecuta, ya que falla con un error de segmentación al leer, tanto con modelos medianos como livianos. Como Tesseract ya tarda 4,2 veces más en la Raspberry Pi que en la PC, es esperable que los tiempos de PaddleOCR resulten excesivos para esta plataforma aun si se lograra ejecutarlo. EasyOCR iguala a Tesseract en precisión en la Raspberry Pi, pero tarda 17,2 s por imagen y usa siete veces más memoria. Se eligió **Tesseract**: es el único que combina una precisión comparable con un tiempo de respuesta y un consumo de memoria compatibles con la Raspberry Pi 3.

Con la cámara Logitech C270 se probó primero el encuadre. La Tabla 6 compara tres distancias entre la cámara y las patentes.

| Distancia | Ancho visto | Ancho de la patente en la imagen | Observación |
|---|---|---|---|
| 20 cm | ≈ 18 cm | ≈ 430 px | Patente pegada al borde de la imagen; enfoque menos nítido. |
| **30 cm** | ≈ 28 cm | ≈ 290-310 px | Margen a ambos lados; el doble del ancho mínimo legible (≈ 120 px). |
| 35 cm | ≈ 32 cm | ≈ 250 px | También legible. |

Tabla 6: Encuadre de la cámara C270 según la distancia a las patentes.

Se eligió 30 cm: las dos plazas entran con margen para que el vehículo quede corrido sin que se corte la patente, y el enfoque fijo de la cámara resulta más nítido que a menor distancia. La cámara queda a la altura de las patentes, de frente y centrada entre las plazas. A partir de una captura en esa posición se definió la zona de cada plaza, y se verificó que cada zona encuentre y lea su patente (Figura 4) y que informe «sin patente» cuando la plaza está vacía.




Figura 4: Captura de la cámara C270 con las zonas de las plazas y las patentes leídas.

La verificación de punta a punta con esta cámara, disparada por los avisos del ESP32 real, se presenta en la Tabla 9.

### 4.1. *Hardware* probado

El estado del *hardware* se resume en la Tabla 7.

| ***Hardware*** | **Estado** |
|---|---|
| Raspberry Pi 3 B+ | Probada: sistema, OCR, base de datos y mensajería funcionando. |
| Pendrive USB | Probado como almacenamiento del sistema. |
| Cámara USB (1.ª unidad) | Defectuosa: no entrega imagen en ningún equipo (Sección 4.4). |
| Cámara USB (2.ª unidad, Jieli 1080p) | Funciona en la PC; incompatible con el USB de la Raspberry Pi 3 (Sección 4.4). Fue devuelta y reemplazada por la C270. |
| Cámara Logitech C270 | Probada en la Raspberry Pi 3: 200 de 200 cuadros completos en todas las configuraciones, tanto en el formato comprimido *Motion-JPEG* (MJPEG) como en el formato sin comprimir de luminancia y color (YUYV). Compatible, ya que ofrece varios modos de transmisión USB (Sección 4.4). Con la cámara conectada la Raspberry Pi no arranca desde el pendrive: se enciende sin la cámara y se la conecta una vez iniciado el sistema. Falta anclarla en la maqueta. |
| Prototipo de maqueta | Tablero con las dos plazas, las patentes impresas montadas sobre cajas y soporte provisorio para la cámara (Figura 5). |

| ***Hardware*** | **Estado** |
|---|---|
| ESP32, HC-SR04 y LED bicolor | Dos HC-SR04 y dos LED funcionando. El ESP32 publica el estado de cada plaza a la Raspberry Pi a través de la red del teléfono móvil. |

Tabla 7: Estado de prueba del hardware.

### 4.2. Planificación del Avance de noviembre

Una vez cerrado el reconocimiento de patentes y la mensajería con el ESP32 real, el Avance de noviembre incorpora el servicio web y la robustez del sistema. Las tareas, en orden cronológico, son:

1. **Cierre de lo pendiente de octubre:** anclaje definitivo de la cámara en la maqueta con nueva calibración de las zonas.
2. **Servicio web (*backend*) y API de consulta:** servicio en Python que atiende la consulta de ubicación por patente y opera sobre la base SQLite, con una interfaz de programación (API) que expone las estadías abiertas (RF06).
3. **Página del tótem:** interfaz construida con las tecnologías estándar de la web, con un campo para ingresar la patente y la visualización de la plaza donde está estacionado el vehículo, con un diseño minimalista orientado al conductor.
4. **Casos límite del OCR:** tratamiento de lecturas de baja confianza o inválidas, de plazas ocupadas sin patente detectable y de caracteres redondos que el reconocimiento confunde (por ejemplo, un 0 o una C leídos como U), exigiendo al menos dos lecturas coincidentes antes de registrar una patente, sin degradar la consistencia del registro (RNF02).
5. **Robustez de la mensajería:** incorporación del mensaje de última voluntad (*Last Will Testament*), para que el *broker* avise cuando el ESP32 se desconecta. La calidad de servicio (QoS 1) y la retención del último estado de cada plaza ya están aplicadas, y la marca de tiempo de cada evento la asigna la Raspberry Pi al registrarlo.
6. **Medición de la latencia y de la consistencia:** medición del tiempo entre el evento físico y la actualización del registro y de la señalización sobre la maqueta (RNF01), y verificación de la coherencia entre el estado sensado, el registro y el LED ante entradas y salidas sucesivas (RNF03).
7. **Tareas opcionales:** incorporación de los formatos de patente de países limítrofes, previa investigación de sus especificaciones, y de un panel de estado general del estacionamiento como evolución del tótem. Se abordan según el tiempo disponible y con menor prioridad que los casos límite del OCR y que el resto de las tareas anteriores.

### 4.3. Documentación de Código

Repositorio: https://github.com/tpII/2026-g2-parkfinder. La carpeta `ocr/` contiene lo que se ejecuta en la Raspberry Pi; `esp32/` contiene el *firmware* del microcontrolador (sensado, LED y publicación MQTT); `banco_pruebas/` contiene los *scripts* de evaluación, que no forman parte del sistema final.

#### 4.3.1. Código que controla el *hardware*

**Cámara (Raspberry Pi), `ocr/captura.py`:** abre la cámara con el sistema de video de Linux (*Video4Linux2*, V4L2) mediante OpenCV [10], fija la resolución y el formato, y descarta los primeros cuadros mientras se ajusta la exposición automática. Antes de cada captura descarta los cuadros acumulados en el *buffer*, para que la imagen corresponda al instante del evento. Con la cámara C270, la apertura demora alrededor de 1,2 s y cada captura unos 200 ms. Las zonas de las plazas se guardan en `config/plazas.json`.

```
class FuenteCamara(FuenteImagen):
    def __init__(self, indice=0, ancho=1280, alto=720, formato="MJPG",
                 calentamiento=10, descarte=2):
        backend = cv2.CAP_DSHOW if os.name == "nt" else cv2.CAP_V4L2
        self._cap = cv2.VideoCapture(indice, backend)
        if not self._cap.isOpened():
            raise ErrorCaptura(f"no se pudo abrir la cámara {indice}")
        self._cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*formato))
        self._cap.set(cv2.CAP_PROP_FRAME_WIDTH, ancho)
        self._cap.set(cv2.CAP_PROP_FRAME_HEIGHT, alto)
        self._cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        ...

    def capturar(self):
        for _ in range(self._descarte):
            self._cap.grab()
        ok, imagen = self._cap.read()
        if not ok or imagen is None:
            raise ErrorCaptura("la cámara no entregó imagen")
        return imagen
```

Una segunda fuente, `FuenteCarpeta`, devuelve fotos de una carpeta con la misma interfaz, lo que permitió probar el sistema completo sin cámara.

**Sensores y LED (ESP32), módulo `sensores`:** el firmware se organiza en tres módulos: `sensores` (HC-SR04 y LED), `comunicacion` (WiFi y MQTT) y el programa principal `parkfinder_esp32.ino`, que es el único que conoce a ambos. El módulo `sensores` mide las plazas por turnos, una cada 100 ms, y aplica dos filtros contra el ruido. El primero es una histéresis: la plaza pasa a ocupada cuando la distancia es de 10 cm o menos y vuelve a libre cuando supera

los 13 cm; si no hay eco, se interpreta como libre. El segundo exige que las lecturas contradigan el estado actual sin interrupción durante 1,5 s, y nunca menos de 5 lecturas, antes de aceptar el cambio. El tiempo de confirmación se mide en milisegundos y no en cantidad de lecturas, para que no dependa de cuántas plazas haya. Al encender, el estado de cada plaza no se supone: se confirma midiendo, con el mismo criterio, y el LED permanece apagado hasta entonces. El LED bicolor indica el estado con rojo para plaza ocupada y verde para libre, y lo maneja el ESP32 localmente según el sensor de su plaza, sin intervención de la Raspberry Pi.

```
static float medirDistanciaCm(const Plaza &p) {
  digitalWrite(p.pinTrig, LOW);   delayMicroseconds(2);
  digitalWrite(p.pinTrig, HIGH);  delayMicroseconds(10);
  digitalWrite(p.pinTrig, LOW);
  unsigned long duracion = pulseIn(p.pinEcho, HIGH, TIMEOUT_US);
  if (duracion == 0) return -1;            // sin eco
  return duracion * 0.0343 / 2.0;          // velocidad del sonido, ida y vuelta
}

static bool lecturaIndicaOcupada(const Plaza &p, float distancia) {
  if (distancia < 0) return false;         // sin eco = no hay objeto cerca
  if (p.ocupada) return distancia <= UMBRAL_LIBERAR_CM;   // 13 cm
  else           return distancia <= UMBRAL_OCUPAR_CM;    // 10 cm
}

// Fragmento de sensoresActualizar(): confirmación por tiempo
if (p.contadorCambio == 0 || lecturaOcupada != p.candidato) {
  p.candidato = lecturaOcupada;            // empieza a contar un posible cambio
  p.contadorCambio = 1;
  p.inicioCambio = millis();
  return false;
}
p.contadorCambio++;
if (p.contadorCambio < LECTURAS_MINIMAS ||
    millis() - p.inicioCambio < TIEMPO_CONFIRMACION_MS) {   // 1500 ms
  return false;                            // todavía no se confirma
}
p.ocupada = p.candidato;                   // cambio confirmado
actualizarLed(p);
```

El fragmento resume `medirDistanciaCm`, `lecturaIndicaOcupada` y `sensoresActualizar`. Los datos de la red WiFi se guardan en un archivo `credenciales.h` que no se sube al repositorio.

### 4.3.2. Código de comunicaciones

La Tabla 8 presenta la estructura de tópicos MQTT acordada entre la Raspberry Pi y el ESP32.

| Tópico | Contenido | Publica |
|---|---|---|
| `parkfinder/plaza/1` | `ocupada` / `libre` | HC-SR04 de la plaza 1 (izquierda vista desde la cámara) |
| `parkfinder/plaza/2` | `ocupada` / `libre` | HC-SR04 de la plaza 2 (derecha) |

Tabla 8: Tópicos MQTT acordados entre la Raspberry Pi y el ESP32.

Cada plaza tiene su propio tópico, numerado según su posición vista desde la cámara: la plaza 1 es la de la izquierda y la plaza 2 la de la derecha. El contenido del mensaje indica únicamente el nuevo estado de la plaza, ocupada o libre, y se publica una sola vez por cada cambio, de modo que la unidad central recibe un aviso por cada vehículo que llega o se retira y no un flujo continuo de lecturas.

**Suscriptor (Raspberry Pi), `ocr/suscriptor.py`:** se suscribe a `parkfinder/plaza/+` mediante un comodín que abarca todas las plazas [3]. Los mensajes recibidos se validan y se encolan, y el hilo principal los atiende de a uno, ya que hay una sola cámara y una lectura no debe bloquear la comunicación con el *broker* Mosquitto [9]. El cliente se reconecta automáticamente ante una caída del *broker* o de la red, y usa una sesión limpia: al conectarse recibe el último estado de cada plaza a través de los mensajes retenidos del *broker*, y no los avisos acumulados mientras estuvo desconectado.

```
def al_recibir(self, _cliente, _datos, mensaje):      # hilo de red de MQTT
    contenido = mensaje.payload.decode("utf-8", errors="replace")
    if not contenido:
        return                                        # mensaje vacío: no es un aviso
    evento = interpretar(mensaje.topic, contenido)
    if evento is None:
        log.warning("mensaje con formato inválido, ignorado: %s %r", mensaje.topic,
contenido)
        return
    self.eventos.put((*evento, time.perf_counter()))

def atender(self, plaza, estado, recibido):           # hilo principal (fragmento)
    if estado == "ocupada":
        time.sleep(self.espera_captura)               # opcional: 0 s con el ESP32
(ya confirma 1,5 s)
        lectura = self.pipeline.procesar_ocupada(plaza)
    else:
        self.pipeline.procesar_libre(plaza)
```

**Publicación (ESP32), módulo `comunicacion`:** utiliza la biblioteca *arduino-mqtt* [11]. Publica en el tópico `parkfinder/plaza/N` el contenido `ocupada` o `libre`, con calidad de servicio (QoS) 1 y con el indicador *retain* activo, solo ante un cambio de estado. El ESP32 se conecta a la red WiFi del teléfono móvil y busca la Raspberry Pi por su nombre (`parkfinder.local`) en cada intento

de conexión, porque la dirección IP cambia con la red. La reconexión no frena la medición: se reintenta cada 5 s, y los cambios ocurridos sin conexión se reenvían al conectar, solo el último estado de cada plaza.

```
// publish(tema, contenido, retain, qos)
bool ok = cliente.publish(tema, contenido, true, 1);
```

**Prueba de integración del ESP32 con la Raspberry Pi.** Antes de contar con el ESP32, el suscriptor se había validado con un simulador de sensores que publica los mismos mensajes (`banco_pruebas/sensor.py`). Luego, con el ESP32 y la Raspberry Pi conectados a la red del teléfono móvil y la cámara C270, se colocó la caja con la patente frente al sensor de cada plaza para simular el ingreso de un vehículo. La Tabla 9 presenta los resultados: las cinco entradas se leyeron y registraron correctamente, y las salidas cerraron las estadías.

| Entrada | Plaza | Patente leída |
|---|---|---|
| 1 | 1 | RE483UK (formato Mercosur) |
| 2 | 2 | MPT236 (formato anterior) |
| 3 | 2 | BJ741CY (formato Mercosur) |
| 4 | 1 | XSO059 (formato anterior) |
| 5 | 2 | MPT236 (formato anterior) |

Tabla 9: Prueba de integración: entradas registradas a partir de los avisos del ESP32.

En las cinco entradas el sistema leyó la patente correcta, y las estadías quedaron registradas y cerradas de forma coherente en la base de datos. Entre que el vehículo llega a la plaza y su registro transcurren cerca de 3,8 s: 1,5 s de confirmación del sensor, 0,3 s de publicación y unos 2,0 s de procesamiento en la Raspberry Pi.

### 4.3.3. Sistema web

No aplica en esta instancia: el servicio web y el tótem están planificados para noviembre.

### 4.3.4. *Software* de la unidad central

***Pipeline*, `ocr/pipeline.py`:** ante una plaza ocupada captura la imagen, recorta la zona de la plaza, lee la patente y la valida. El siguiente fragmento muestra ese tramo.

```
imagen = self.fuente.capturar()
zona = escalar_zona(self.plazas[plaza], self.resolucion, imagen)
recorte = recortar_plaza(imagen, zona)
if recorte is not None:              # si no, no se encontró una patente en la zona
```

```
    texto, _ = self.motor.leer(recorte)
    patente = corregir(texto)        # corrección dígito/letra por posición
    formato = validar(patente)       # formato anterior o Mercosur
```

La validación utiliza expresiones regulares para el formato anterior (tres letras y tres números) y el formato Mercosur (dos letras, tres números y dos letras) [12]. Antes de registrar un ingreso, el *pipeline* espera a que el reloj del sistema esté sincronizado por *Network Time Protocol* (NTP), ya que la Raspberry Pi no tiene un reloj con batería.

**Recorte, `ocr/recorte.py`:** dentro de la zona de la plaza se buscan manchas oscuras con forma de carácter (más altas que anchas), de altura similar y alineadas en una fila; el grupo más grande es la patente. También se incorporan las manchas más anchas que altas y alineadas con la fila, que corresponden a caracteres que la cámara unió (por ejemplo, la R y la E). Si no hay al menos 4 caracteres alineados, se considera que no hay patente.

**Motor, `ocr/motores/motor_tesseract.py`:** preprocesa la imagen (escala de grises y filtro de mediana) y ejecuta Tesseract con `--psm 8` (una palabra) y una lista blanca de caracteres A-Z y 0-9, en una sola llamada que devuelve el texto.

**Persistencia, `ocr/base_datos.py`:** dos tablas SQLite. `estadias` guarda plaza, patente y hora de ingreso y de egreso; las estadías sin egreso son las plazas ocupadas y es la tabla que consultará el tótem. `lecturas` guarda el resultado de cada captura, para diagnóstico.

### 4.4. Problemas y soluciones

La Tabla 10 reúne los problemas encontrados y la solución o alternativa adoptada.

| **Problema** | **Solución o alternativa** |
|---|---|
| Cámara USB 1.ª unidad: no entrega imagen en ningún equipo (se detecta, pero sin datos de video). | Falla de *hardware*, informada al docente. Se reemplazó. |
| Cámara USB 2.ª unidad (chip Jieli, 1080p): funciona en la PC, pero en la Raspberry Pi ningún cuadro llega completo (0 de 40). Utiliza un único modo de transferencia isócrona, el más exigente de USB 2.0, que el controlador USB de la Raspberry Pi 3 no sostiene. Se descartaron la alimentación, el formato de video, el programa de captura y la configuración del controlador. | La cámara se devolvió y se reemplazó por una Logitech C270, que ofrece siete modos de transmisión USB (el controlador elige el menor que alcanza). En la Raspberry Pi entregó 200 de 200 cuadros completos. La captura es intercambiable, por lo que solo cambió la fuente de imagen. |
| PaddleOCR falla con error de segmentación en la Raspberry Pi. EasyOCR tarda 17 s por imagen, usa 774 MB de RAM y sobrecalienta el procesador. | Se descartaron a favor de Tesseract. |

| **Problema** | **Solución o alternativa** |
|---|---|
| Baja tensión y reinicio espontáneo de la Raspberry Pi alimentada con un cargador de celular. | Se reemplazó por una fuente nueva; desde entonces no se registró baja tensión. |
| Tras el reinicio, el reloj arrancó 2,6 horas atrasado (la Raspberry Pi no tiene reloj con bat ería). | El *pipeline* espera la sincronización NTP antes de registrar eventos. |
| Con la cámara C270 conectada, la Raspberry Pi no arranca desde el pendrive. | Se enciende sin la cámara y se la conecta una vez iniciado el sistema; después funciona con normalidad. |
| El sensor de la plaza 2 devolvía siempre «sin eco». | Se rehízo su cableado y ahora mide correctamente, con el Echo en el GPIO 34. |
| Al encender, el ESP32 publicaba «libre» sin haber medido; con un vehículo estacionado, la Raspberry Pi registraba una salida y una entrada falsas. | El estado inicial se confirma midiendo (1,5 s) antes de publicarlo, y el LED queda apagado hasta entonces. |
| La confirmación por cantidad de lecturas tardaba 3 s con dos plazas, porque se miden por turnos. | La confirmación se mide por tiempo (1,5 s, con al menos 5 lecturas), sin depender de la cantidad de plazas. |
| El ESP32 buscaba al *broker* por una dirección IP fija, que cambia con la red. | Busca a la Raspberry Pi por su nombre en la red (`parkfinder.local`). |
| El suscriptor, con sesión persistente, recibió 10 avisos acumulados durante 13 minutos y abrió estadías falsas. | Pasó a sesión limpia: el estado llega por los mensajes retenidos, que dan el último estado real de cada plaza. |
| Al reordenar la maqueta, las cajas acercadas a los sensores dejaron de coincidir con las zonas de las plazas. | Se reubicó la cámara a 30 cm de las patentes y se verificó que las zonas coincidieran, sin recalibrar. |

Tabla 10: Problemas encontrados y soluciones o alternativas.

## 5. Documentación Relacionada

**Fotos y capturas:** la Figura 3 (sección 3) muestra el montaje del ESP32 con los dos sensores y los dos LED, y la Figura 4 (sección 4) una captura de la cámara C270 con las zonas de las plazas y la lectura de cada patente. La Figura 5 muestra, de izquierda a derecha y de arriba hacia abajo, el prototipo de maqueta, la cámara Logitech C270 en su soporte provisorio y la Raspberry Pi con la cámara y el pendrive conectados.




Figura 5: Maqueta, cámara con soporte provisorio y Raspberry Pi con pendrive.

**Enlaces adicionales:**

- Repositorio de código: https://github.com/tpII/2026-g2-parkfinder
- Bitácora: https://github.com/tpII/2026-g2-parkfinder/wiki
- Video del Plan de Proyecto: https://youtu.be/3SANUAXyRSI

## Bibliografía

[1] SparkFun Electronics, «HC-SR04 Ultrasonic Ranging Module — Datasheet». Accedido: 3 de octubre de 2026. [En línea]. Disponible en: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[2] Espressif Systems, «ESP32 — Technical Documentation». Accedido: 3 de octubre de 2026. [En línea]. Disponible en: https://www.espressif.com/en/products/socs/esp32

[3] OASIS, «MQTT Version 3.1.1 — OASIS Standard». Accedido: 5 de octubre de 2026. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[4] Raspberry Pi Ltd, «Raspberry Pi Documentation». Accedido: 1 de octubre de 2026. [En línea]. Disponible en: https://www.raspberrypi.com/documentation/

[5] SQLite Consortium, «SQLite Documentation». Accedido: 5 de octubre de 2026. [En línea]. Disponible en: https://www.sqlite.org/docs.html

[6] Tesseract OCR Project, «Tesseract OCR — Documentation». Accedido: 22 de septiembre de 2026. [En línea]. Disponible en: https://tesseract-ocr.github.io/

[7] JaidedAI, «EasyOCR — Ready-to-use OCR with 80+ supported languages». Accedido: 22 de septiembre de 2026. [En línea]. Disponible en: https://github.com/JaidedAI/EasyOCR

[8] PaddlePaddle, «PaddleOCR — Awesome multilingual OCR toolkits based on PaddlePaddle». Accedido: 22 de septiembre de 2026. [En línea]. Disponible en: https://github.com/PaddlePaddle/PaddleOCR

[9] Eclipse Foundation, «Eclipse Mosquitto — An open source MQTT broker». Accedido: 5 de octubre de 2026. [En línea]. Disponible en: https://mosquitto.org/documentation/

[10] OpenCV Team, «OpenCV Documentation». Accedido: 21 de septiembre de 2026. [En línea]. Disponible en: https://docs.opencv.org/

[11] Joël Gähwiler, «arduino-mqtt — MQTT library for Arduino». Accedido: 6 de octubre de 2026. [En línea]. Disponible en: https://github.com/256dpi/arduino-mqtt

[12] G. M. C. MERCOSUR, «Resolución GMC N.º 33/14: Patente y sistema de consultas sobre vehículos del MERCOSUR». Accedido: 7 de octubre de 2026. [En línea]. Disponible en: https://www.argentina.gob.ar/normativa/nacional/norma-244472/texto